

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Output obtained:

- Raw Training Data Shape: (60000, 28, 28)
- Raw Testing Data Shape: (10000, 28, 28)

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Output obtained:

- Preprocessed Training Data Shape: (60000, 784)
- Preprocessed Testing Data Shape: (10000, 784)

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

`model.summary()` output: Total params 109,386 (427.29 KB) – all trainable.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Trained for 20 epochs, batch size 32. Final epoch (20/20): training accuracy 0.9392, training loss 0.1600, validation accuracy 0.8826, validation loss 0.4132.

Task 5: Model Evaluation

Baseline test accuracy: **0.8805**. Full precision/recall/F1 per class obtained via `classification_report`; macro avg precision/recall/F1 all ≈ 0.88 . See Performance Comparison table in Section 9 for consolidated figures.

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

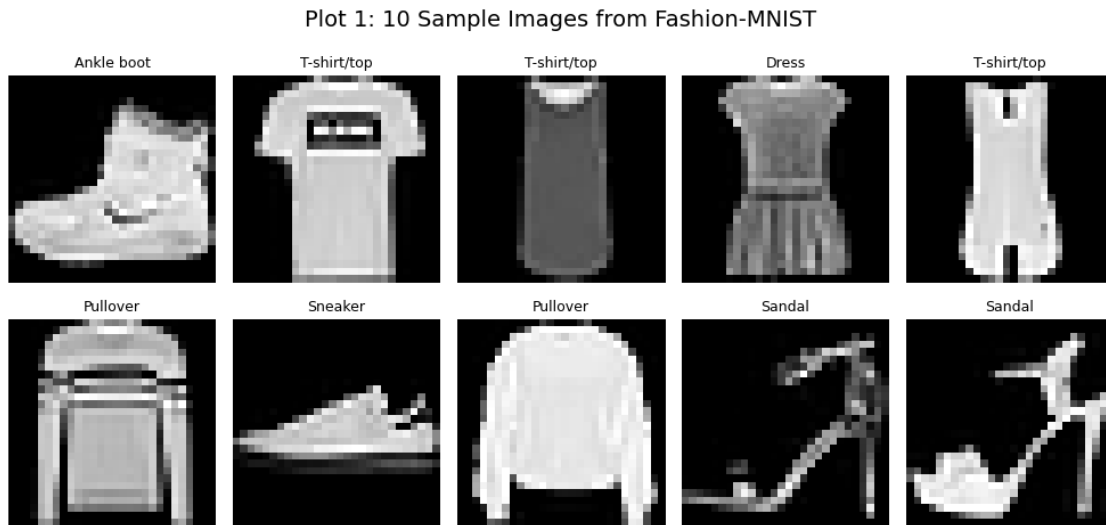
1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

*Note: the run used RandomizedSearchCV with **3-fold** cross-validation and 5 candidate configurations (a narrowed search, scaled down from the manual's 5-fold spec for execution-time reasons). If your grading criteria require literal 5-fold CV, re-run the search with `cv=5` before submitting.*

8. Mandatory Plots

Generate the following plots. Write a 2–3 line inference underneath each.

1. Sample Images



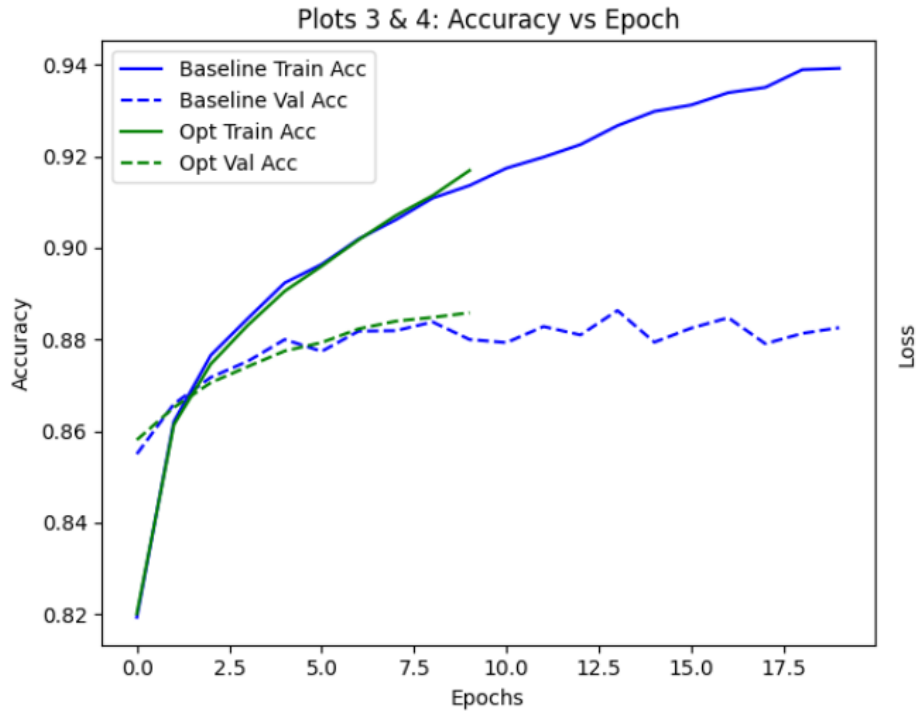
The 10 sample images span the various types of garments observed in the dataset for analysis.

2. Class Distribution



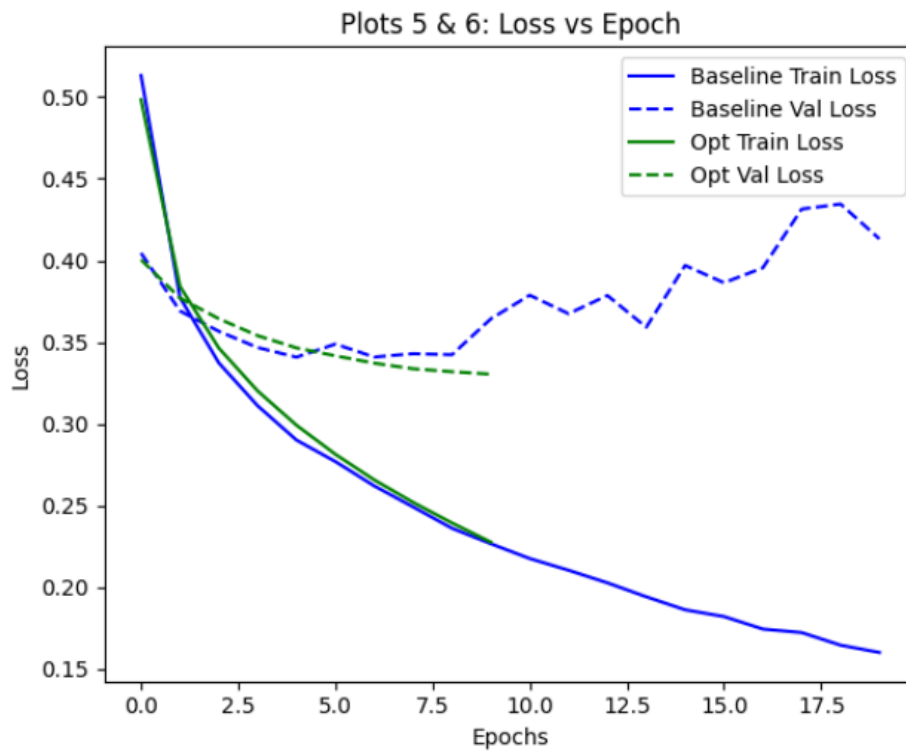
Each of the 10 classes has 6000 training images, for sake of balance.

3 & 4. Training Accuracy vs Epoch / Validation Accuracy vs Epoch



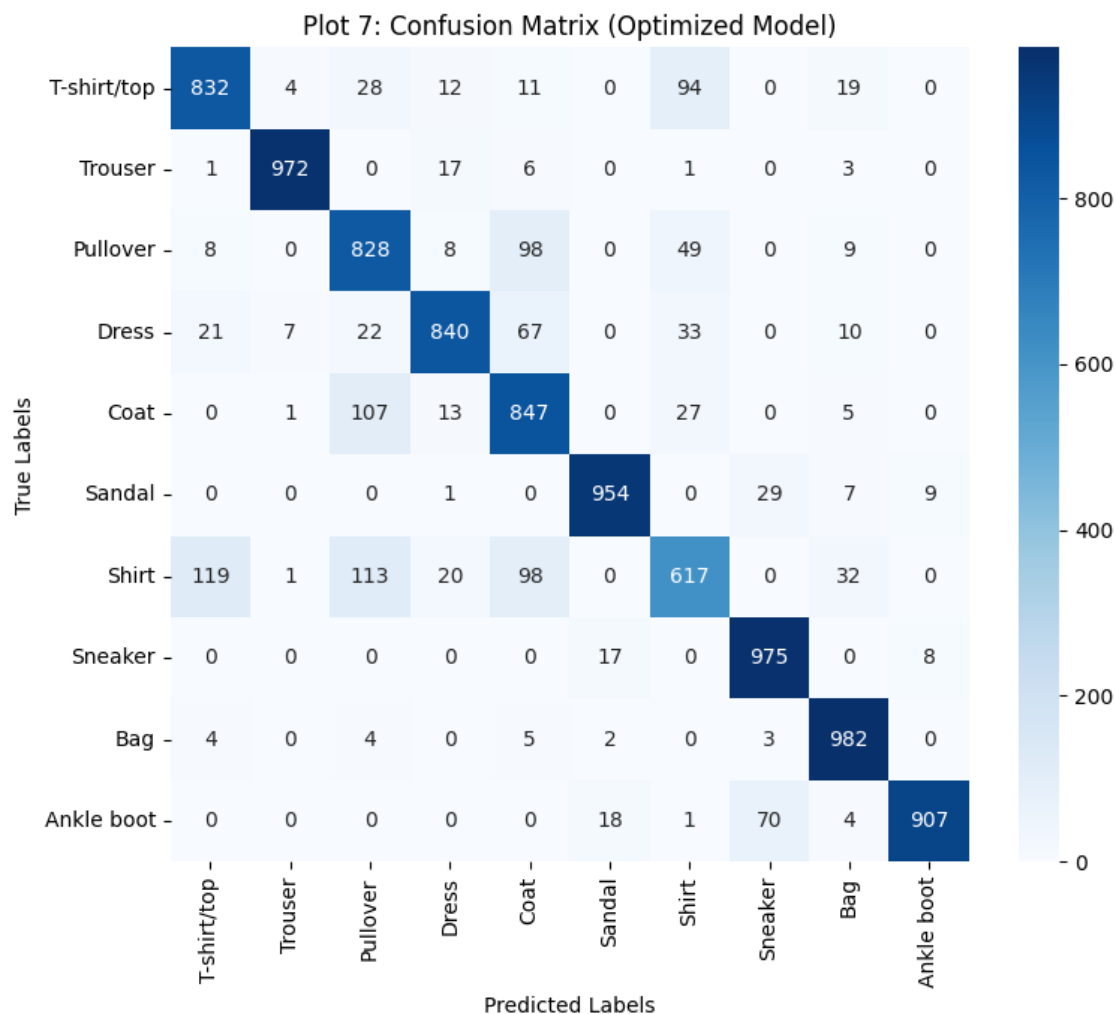
Accuracy climbs to 0.94 by the 20th iteration, while accuracy stagnates at 0.88 at epoch 5, after which it oscillates.

5 & 6. Training Loss vs Epoch / Validation Loss vs Epoch



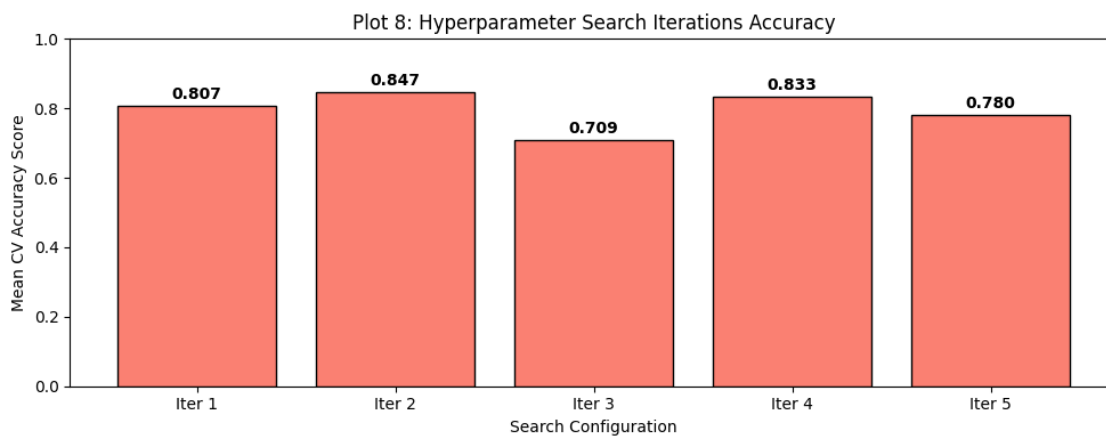
Baseline training loss falls to 0.16, validation loss reaches 0.34 at around epoch 9. It then rises to 0.42 which shows overfitting. The optimised model's validation loss still decreases.

7. Confusion Matrix



Shirt is a hard class to classify with only 617 correct ones. Pullover and coat are somewhere in the middle. Trousers, bag, etc are classified almost perfectly.

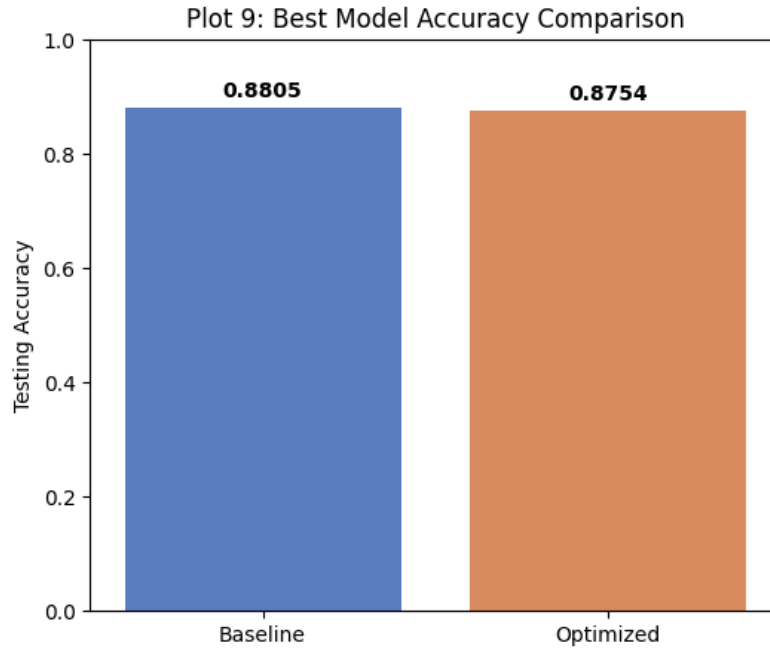
8. Hyperparameter Search Results



Mean CV accuracy across the 5 configurations ranges from 0.7 to 0.85. 1 layer. 256 neurons, tanh,

Adam, learning rate 0.001 was found to be the best.

9. Best Model Accuracy Comparison



Optimised model scores 0.8754 compared to baseline model's 0.8805 i.e. a faster model was found but it was slightly worse than the baseline model. The time saved is significant though.

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	Adam
Activation Function	Tanh
Epochs	10
Dropout	0.0
Cross-validation Accuracy	0.8473
Testing Accuracy	0.8754

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8805	0.8754
Precision	0.8831	0.8770
Recall	0.8805	0.8754
F1-score	0.8811	0.8745
Training Time	162.78s	59.28s

10. Discussion

1. Which optimization method was used?

RandomizedSearchCV with 3 fold cross validation over 5 sampled configurations

2. Which hyperparameters were selected?

1 hidden layer, 256 neurons, learning rate 0.001, batch size 64, Adam optimizer, Tanh activation, 10 epochs.

3. Did optimization improve performance?

The optimized model scored 0.8754 vs the baseline's 0.8805 test accuracy, a slight decrease. However, what improved was training efficiency: 59.28s vs 162.78s, significantly faster

4. Which hyperparameter had the greatest impact?

Using only 1 hidden layer (vs the baseline's 2) and still landing within half a point of baseline accuracy, hidden layer depth appears to trade off against training time more than it costs accuracy for this dataset.

5. Compare Grid Search and Randomized Search.

Grid Search exhaustively evaluates every combination in the search space, guaranteeing the best result within the grid. Randomized Search samples a fixed number of combinations at random. It is less exhaustive but takes a lot less time.

6. Recommend the best MLP configuration.

1 hidden layer of 256 neurons with Tanh activation, Adam optimizer at learning rate 0.001, batch size 64, trained for 10 epochs with no dropout.

11. Experiment 2 - Extension: XOR Gate using a Multi Layer Perceptron

The XOR gate is not linearly separable, so a single layer perceptron can never converge on it (no straight line can separate the two classes). In this extension, a Multi Layer Perceptron, trained from scratch using backpropagation and gradient descent, was used to learn the XOR function instead.

Training Data:

$$(0, 0) \rightarrow 0, (0, 1) \rightarrow 1, (1, 0) \rightarrow 1, (1, 1) \rightarrow 0$$

1. Training Log

Epoch	Loss (MSE)	Predictions	Target
1	0.25087	[0, 1, 0, 1]	[0, 1, 1, 0]
500	0.09661	[0, 1, 1, 0]	[0, 1, 1, 0]
1000	0.01005	[0, 1, 1, 0]	[0, 1, 1, 0]
1500	0.00412	[0, 1, 1, 0]	[0, 1, 1, 0]
2000	0.00245	[0, 1, 1, 0]	[0, 1, 1, 0]
2500	0.00169	[0, 1, 1, 0]	[0, 1, 1, 0]
3000	0.00128	[0, 1, 1, 0]	[0, 1, 1, 0]
3500	0.00102	[0, 1, 1, 0]	[0, 1, 1, 0]
4000	0.00084	[0, 1, 1, 0]	[0, 1, 1, 0]
4500	0.00071	[0, 1, 1, 0]	[0, 1, 1, 0]
5000	0.00062	[0, 1, 1, 0]	[0, 1, 1, 0]

Final Result

Final Predictions	[0, 1, 1, 0]
Target Labels	[0, 1, 1, 0]
Accuracy	100.0%

The loss decreases sharply within the first ~ 500 –1000 epochs and then flattens out, with predictions matching the target labels exactly from epoch 500 onward - unlike a single layer perceptron, which would not reach convergence given such a problem.

2. Decision Boundary Evolution

The plots below track how the MLP's decision boundary changes over training. Since the network has a hidden layer, the boundary is a curve rather than a single straight line, which is precisely what allows it to separate XOR.

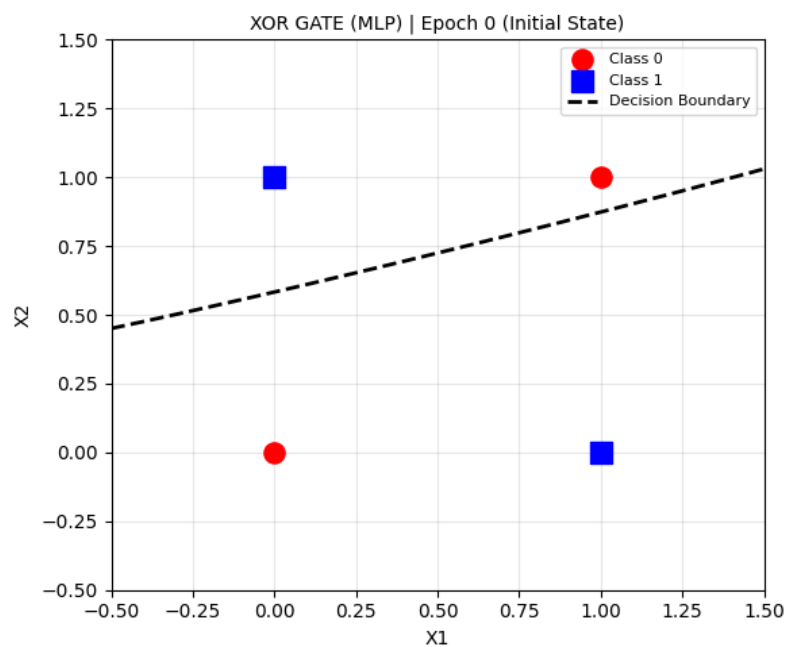


Figure 1: Initial (Untrained) Decision Boundary – Epoch 0

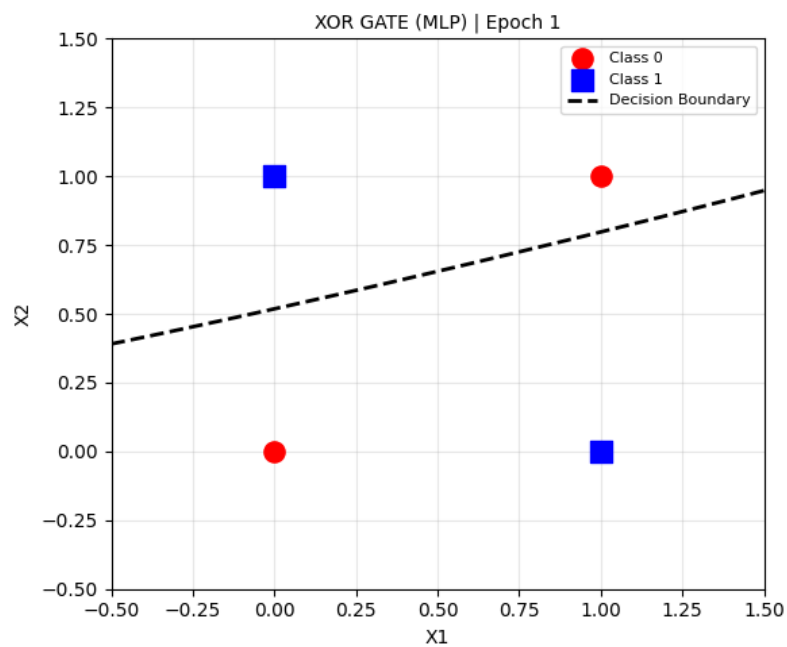


Figure 2: Decision Boundary after Epoch 1

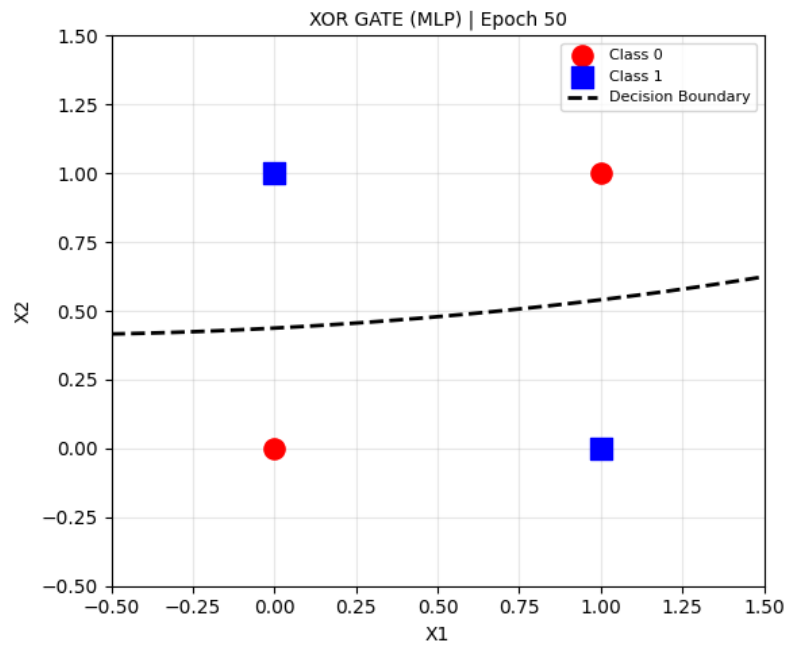


Figure 3: Decision Boundary at Epoch 50

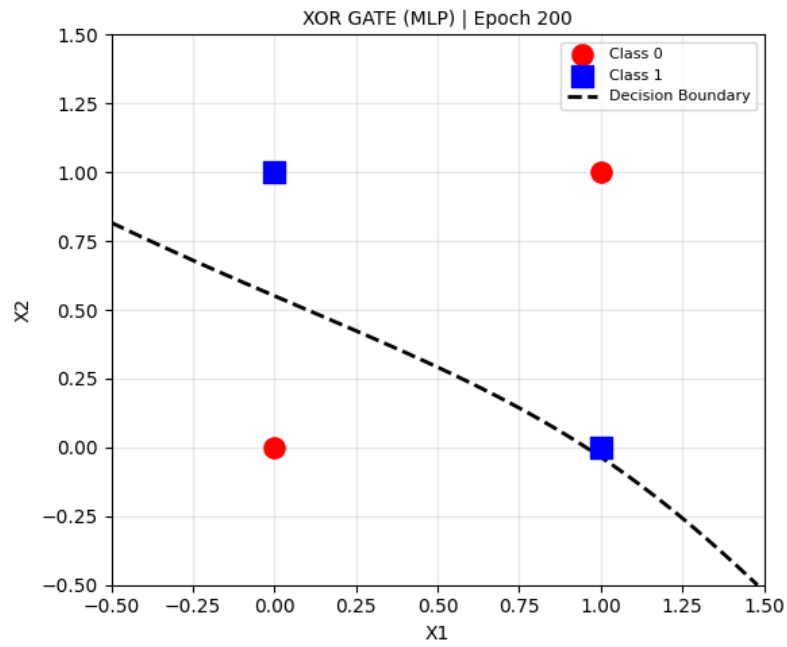


Figure 4: Decision Boundary at Epoch 200

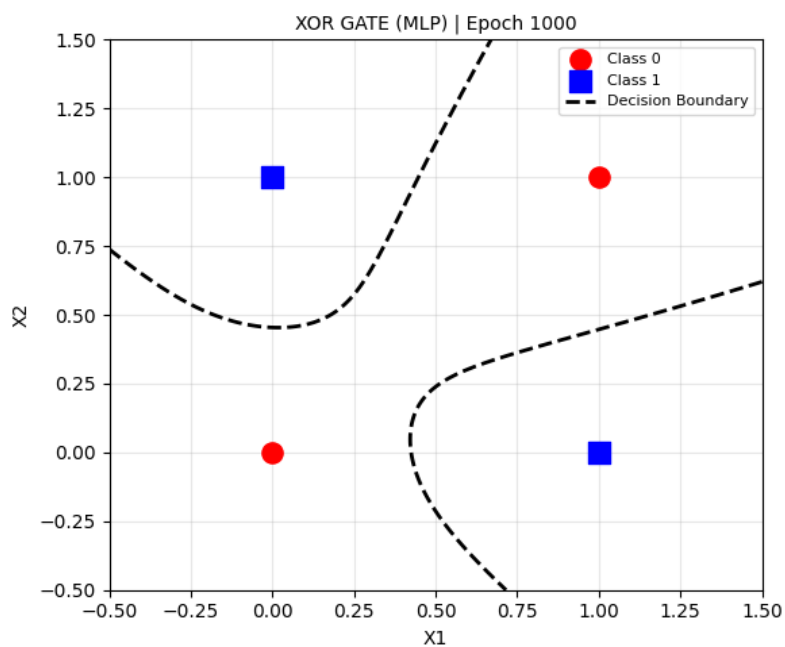


Figure 5: Decision Boundary at Epoch 1000

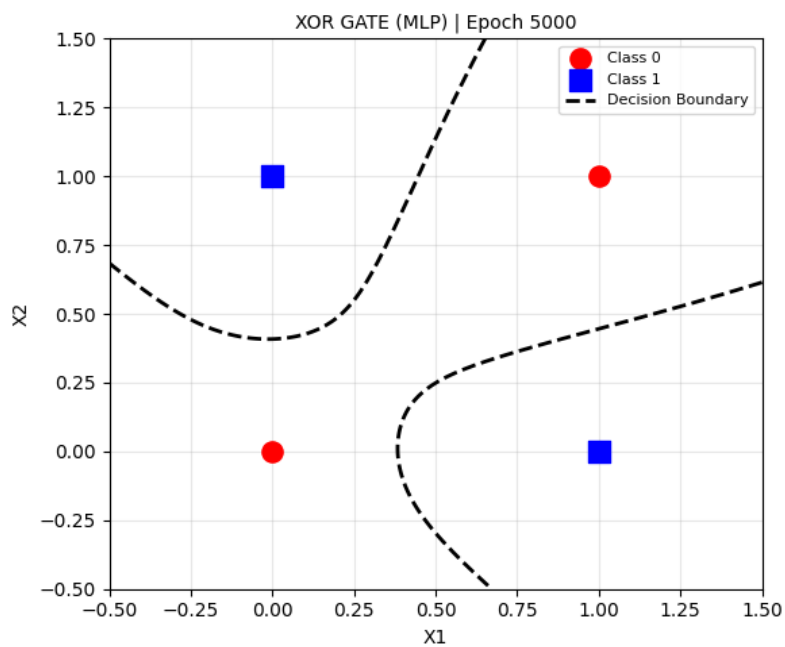


Figure 6: Final Decision Boundary at Epoch 5000

3. Training Loss Curve

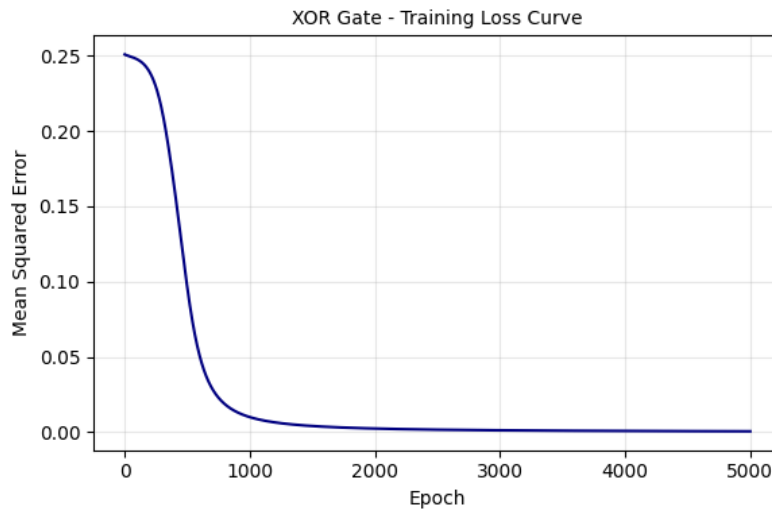


Figure 7: Training Loss (MSE) vs Epoch

Observation: The loss drops steeply between roughly epoch 200 and 1000. This is exactly the window in which the decision boundary visibly bends from a near straight line into the curve that separates the two XOR classes.

4. Observations

- A single layer perceptron cannot solve XOR because the problem is not linearly separable; a multi layer perceptron overcomes this by learning a hidden representation that reshapes the input space, allowing a nonlinear decision boundary.
- The decision boundary starts out nearly linear, and only develops its characteristic curve as training progresses past a few hundred epochs.
- The network achieves 100% accuracy before the full 5000 epochs, though loss continues to decrease slightly further.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.